Tribhuvan University
Institute of Science and Technology

St. Xavier's College, Maitighar, Kathmandu

Project Proposal

on

HimalayaAir: A Real-Time Air Quality Intelligence Platform for the Kathmandu Valley

---

Submitted by:

Dipan Kharel
Roll No.: 79010355
B.Sc. CSIT, 7th Semester

Submitted to:

Department of Computer Science
St. Xavier's College, Maitighar
Tribhuvan University

February 2026

Contents

List of Figures

List of Tables

1. Introduction

Kathmandu Valley, the political and economic heart of Nepal, sits inside a bowl-shaped terrain at approximately 1,300 metres above sea level. This geography, combined with a rapidly growing vehicle fleet, brick kiln operations, seasonal biomass burning, and festival-related pollution events, traps particulate matter at dangerously high concentrations for extended periods. Annual average $PM_{2.5}$ in the valley frequently exceeds the World Health Organisation guideline of 5 $\mu g/m^3$ by a factor of fifteen or more during the dry winter season [1].

This project proposes **HimalayaAir** — a full-stack real-time data engineering platform that ingests live air quality sensor readings from global monitoring networks, enriches them with meteorological and satellite-derived fire data, processes them through a distributed streaming pipeline, and presents the results on an interactive dashboard tailored specifically to Kathmandu Valley. The platform demonstrates all five data engineering curriculum topics mandated by Tribhuvan University for the B.Sc. CSIT final year project: data modelling and database systems, distributed systems and big data processing, pipeline orchestration, infrastructure management, and advanced programming.

The system ingests data from the OpenAQ v3 API every five minutes, enriches readings with Open-Meteo weather data and NASA FIRMS fire-detection events, processes them through Apache Kafka and Spark Structured Streaming, computes EPA-standard Air Quality Index (AQI) values, stores results in a TimescaleDB hypertable, and serves them through a FastAPI backend and a React 18 frontend with live WebSocket updates, a 72-hour SARIMA forecast, and IDW spatial interpolation across a 50×50 grid.

2. Problem Statement

Kathmandu Valley is consistently ranked among the twenty most polluted urban areas in the world [2]. Residents, policymakers, and public health professionals urgently need accessible, real-time tools to understand and respond to local air quality conditions. Existing global platforms such as IQAir provide bare AQI numbers for Kathmandu but suffer from several critical limitations:

1. **No local context.** No platform provides Kathmandu-specific seasonal analysis, Tihar festival event correlation, or recognition of the valley's unique bowl-shaped terrain that traps pollutants.

2. **No spatial interpolation.** Existing platforms display station-level point data only. No freely available tool offers valley-wide AQI estimation at locations between monitoring stations.

3. **No local forecasting.** No freely accessible platform provides short-term AQI forecasting for Kathmandu incorporating local meteorological covariates and seasonal models tuned to the valley's pollution cycles.

4. **No pipeline transparency.** Residents and researchers have no visibility into data freshness, sensor health, or pipeline status.

5. **No historical depth.** Understanding how Kathmandu's air quality has evolved since 2022, and how specific events have affected it, requires a multi-year historical archive with event annotation capability that no existing free platform provides.

HimalayaAir is built to fill this gap with a locally tailored, open-source, and fully self-hosted platform.

3. Objectives

The following objectives are defined using the SMART framework (Specific, Measurable, Achievable, Relevant, and Time-bound), scoped to a 10–12 week development timeline.

1. To design and implement a real-time data ingestion pipeline that polls the OpenAQ v3 API every five minutes for Kathmandu Valley monitoring stations, publishing structured messages to Apache Kafka with end-to-end latency under ten minutes from sensor measurement to dashboard display.

2. To build an Apache Spark Structured Streaming job that computes EPA 2024-standard AQI for five pollutants ($PM_{2.5}$, PM_{10}, NO_2, O_3, CO), assigns district-level spatial context via PostGIS, detects statistical anomalies, and persists results to a TimescaleDB hypertable.

3. To orchestrate five Apache Airflow DAGs covering historical backfill (2022–present), weather enrichment, 72-hour SARIMA forecasting, automated data quality checks, and daily NASA FIRMS fire-event loading.

4. To develop a FastAPI backend exposing ten REST endpoints and one WebSocket endpoint, with P95 response latency under 200 ms at 20 concurrent users.

5. To deliver a React 18 single-page application featuring a full-screen Mapbox GL live map, Recharts charts, a D3.js calendar heatmap spanning three years of historical data, a 72-hour forecast panel, and a demo mode — fully functional on a personal laptop running Docker Compose with zero cloud spend.

Table 3.1: SMART Criteria Verification for Project Objectives

Obj.	Description	S	M	A	R	T
1	Real-time Kafka ingestion, $<$10 min latency	✓	✓	✓	✓	✓
2	Spark AQI computation, anomaly detection	✓	✓	✓	✓	✓
3	5 Airflow DAGs, historical backfill	✓	✓	✓	✓	✓
4	FastAPI 10 REST + WebSocket, P95 $<$200ms	✓	✓	✓	✓	✓
5	React SPA with map, charts, demo mode	✓	✓	✓	✓	✓

4. Methodology

4.1 Requirement Identification

4.1.1 Study of Existing Systems and Literature Review

The literature review spans four areas relevant to HimalayaAir: air quality monitoring, data engineering platforms, analytical methods, and Kathmandu-specific pollution studies.

Kumar et al. [3] demonstrated that dense low-cost sensor networks can provide spatial resolution unavailable from regulatory-grade stations, provided calibration and anomaly-detection pipelines are applied — directly motivating HimalayaAir's use of OpenAQ community sensors with z-score anomaly filtering. Badura et al. [4] evaluated low-cost optical particle counters, finding that site-specific calibration significantly improves accuracy. IQAir [2] and WHO [1] data confirm that Kathmandu's mean annual $PM_{2.5}$ exceeds $46\,\mu g/m^3$, underscoring the urgency of a locally tailored tool.

On the data engineering side, Kreps et al. [5] introduced Apache Kafka as a high-throughput, fault-tolerant messaging system whose publish-subscribe architecture is the standard choice for decoupling sensor ingestion from processing. Zaharia et al. [6] presented Apache Spark with Structured Streaming providing exactly-once semantics over continuous data. Khelifati et al. [7] benchmarked TimescaleDB, showing its hypertable and continuous aggregate abstractions accelerate time-range queries by orders of magnitude over full-table scans. Harenslak and de Ruiter [8] provide the reference on Airflow DAG design patterns used to orchestrate HimalayaAir's five pipeline workflows.

For AQI and analytics, the EPA [9] provides the definitive 2024 breakpoint table implemented for all AQI computations. Apte et al. [10] established the epidemiological link between $PM_{2.5}$ and ischemic heart disease mortality, contextualising the health advisory feature. Shepard [11] introduced IDW spatial interpolation applied here across a 50×50 valley grid; Li and Heap [12] confirmed IDW is appropriate when station density is low and data is non-Gaussian. Box and Jenkins [13] established the SARIMA models underpinning the 72-hour forecasting module, and Bhatti et al. [14] demonstrated that meteorological covariates significantly reduce $PM_{2.5}$ forecast error.

Kathmandu-specific studies provide essential domain context. Putero et al. [15] characterised the influence of open biomass burning on $PM_{2.5}$ in the valley's pre-monsoon season, motivating the NASA FIRMS fire-event overlay. Sharma et al. [16] identified vehicular exhaust, road dust, biomass combustion, and brick kiln emissions as dominant $PM_{2.5}$ sources. Giri et al. [17] documented a steady air quality worsening trend from 2000 to 2012 corre-

lated with urbanisation, providing the historical baseline for the HimalayaAir archive.

4.1.2 Requirement Analysis

Functional Requirements

Table 4.1: Functional Requirements

Req. ID	Description	Inputs	Outputs
FXNR_1	Poll OpenAQ v3 API every 5 minutes for Kathmandu Valley stations.	Scheduled timer, station IDs	Raw Kafka messages on `raw-aq-readings`
FXNR_2	Compute EPA 2024 AQI for $PM_{2.5}$, PM_{10}, NO_2, O_3, CO.	Pollutant concentration, unit	Integer AQI, health category, colour code
FXNR_3	Assign each reading to a Kathmandu district via PostGIS spatial join.	Station coordinates, district polygons	`district_id` on each reading row
FXNR_4	Flag anomalous readings where z-score exceeds 3.0 against the 7-day rolling mean.	7-day hourly aggregate, current reading	`is_anomaly = TRUE` flag
FXNR_5	Broadcast live station updates via WebSocket within 30 seconds of new data.	Processed Kafka messages	WebSocket JSON broadcast to all clients
FXNR_6	Compute IDW spatial interpolation on a 50×50 grid.	Active station AQI values	2500-element integer array
FXNR_7	Generate 72-hour SARIMA forecasts with 80% confidence intervals hourly.	90-day hourly data, weather covariates	72 forecast rows per station
FXNR_8	Historical backfill of OpenAQ data from 2022-01-01 idempotently.	Date range, OpenAQ API	$>$100,000 rows in `aq_readings`
FXNR_9	Load NASA FIRMS fire-detection events daily for South Asia.	FIRMS CSV, MAP_KEY	Rows in `fire_events` table
FXNR_10	Expose a pipeline health endpoint reporting status of all components.	Live DB check, run logs	JSON with component statuses
FXNR_11	Provide historical ex-	Date range, station,	Paginated time-series

Non-Functional Requirements

Table 4.2: Non-Functional Requirements

Req. ID	Category	Description	Target
NFXNR_1	Performance	REST endpoints respond within 200 ms at P95 for 20 concurrent users.	Verified by locust load test
NFXNR_2	Latency	End-to-end pipeline latency under 10 minutes.	Measured sensor timestamp vs. broadcast time
NFXNR_3	Scalability	Sustain 20 concurrent WebSocket connections without dropped messages.	FastAPI `ConnectionManager`
NFXNR_4	Data Integrity	All writes use `ON CONFLICT DO NOTHING` to prevent duplicates.	`UNIQUE(station_id, pollutant, timestamp)`
NFXNR_5	Reliability	Retry failed API calls up to 3 times with exponential backoff; route to DLQ on failure.	Backoff: 30s, 60s, 120s
NFXNR_6	Maintainability	All schema changes managed via Alembic migrations with `downgrade()` implemented.	Zero raw SQL outside migrations
NFXNR_7	Observability	All services use `structlog` with JSON output; no bare `print()` statements.	Verified by grep at each phase
NFXNR_8	Portability	Complete system starts with a single `docker compose up -d` on any 8 GB machine.	9-service health check script
NFXNR_9	Cost	Total cloud and API cost shall not exceed USD 20 over the project lifetime.	All primary sources free
NFXNR_10	Security	No API keys committed to version control; all secrets via environment variables.	`.env` listed in `.gitignore`
NFXNR_11	Compatibility	Frontend usable at a minimum viewport width of 375 px. 9	Tested on Chrome, Firefox, mobile
NFXNR_12	Memory	Complete Docker Compose stack runs within an	Memory limits set per service

4.2 Feasibility Study

4.2.1 Technical Feasibility

HimalayaAir is built entirely on open-source, production-grade technologies: Docker Compose, TimescaleDB with PostGIS, Apache Kafka, Apache Spark 3.5, Apache Airflow 2.9, FastAPI, React 18, and Mapbox GL. All data sources — OpenAQ v3, Open-Meteo, NASA FIRMS, and GADM boundary files — are free and require no paid subscription. The developer has completed relevant B.Sc. CSIT coursework in Python, databases, distributed computing, and web development. The project is therefore technically feasible.

4.2.2 Operational Feasibility

The system is operated entirely through a web browser after a single `docker compose up` command, requiring no specialised hardware or user accounts. The live dashboard, demo mode, and pipeline health panel make the system comprehensible to non-technical stakeholders. Dead-letter queues, anomaly detection, and automated data quality DAGs provide operational resilience. The project is therefore operationally feasible.

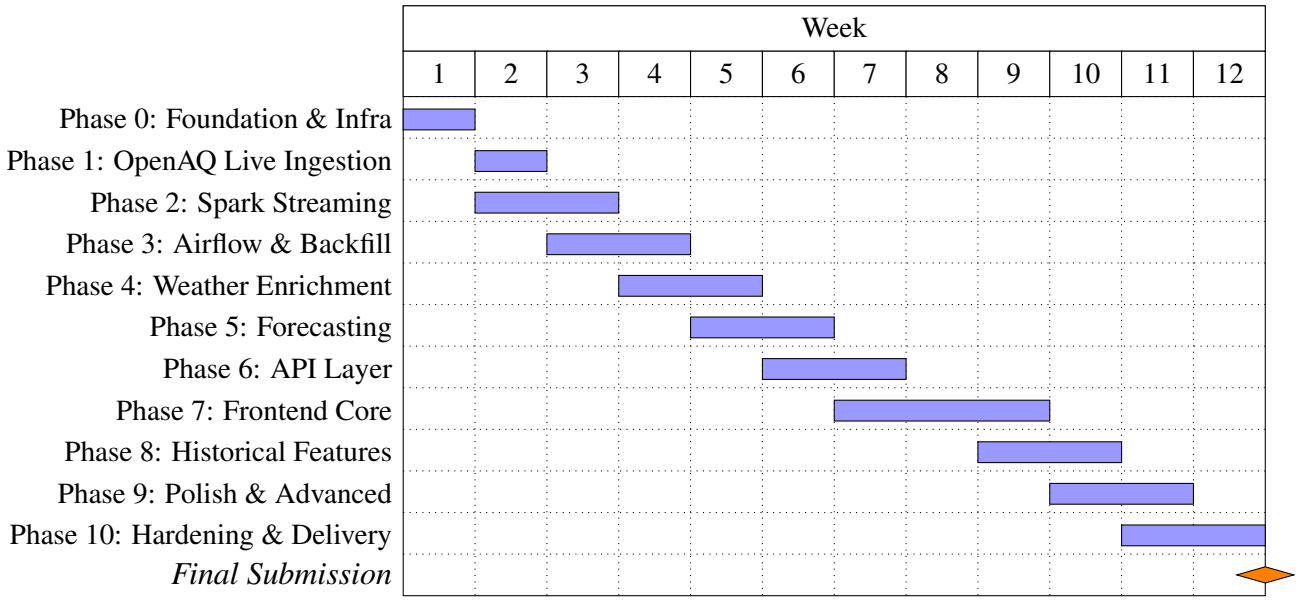
4.2.3 Economic Feasibility

The total monetary cost is zero to a maximum of USD 20 over the project lifetime. All software is open-source, all data sources are free-tier, and development occurs on the student's personal laptop, eliminating cloud compute costs. The project is therefore economically feasible.

4.2.4 Schedule Feasibility

The project is planned across ten phases over a 12-week period. Figure 4.1 presents the development schedule.

Figure 4.1: Project Development Schedule — Gantt Chart

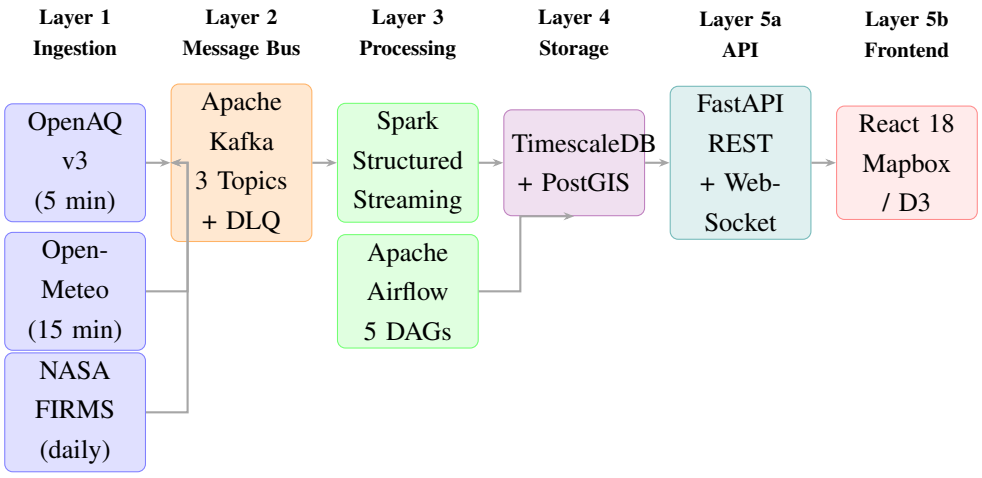


4.3 High Level Design of System

4.3.1 System Architecture Overview

HimalayaAir is structured as a five-layer data engineering pipeline illustrated in Figure 4.2. Data flows unidirectionally from ingestion through the message bus, processing, storage, and finally to the API and presentation layers.

Figure 4.2: HimalayaAir System Architecture



4.3.2 System Description

The **ingestion layer** comprises two always-on Docker services: an `openaq-poller` querying the OpenAQ v3 API every five minutes and a `weather-poller` querying Open-Meteo

every fifteen minutes, both publishing JSON messages to Kafka with exponential back-off retry and dead-letter queuing. NASA FIRMS fire-detection data is loaded daily by an Airflow DAG.

The **message bus** (Apache Kafka) decouples ingestion from processing via four topics: `raw-aq-readings` (3 partitions), `weather-data`, `processed-aq-readings`, and a dead-letter topic. The **processing layer** (Spark Structured Streaming) consumes readings in five-minute micro-batches, computing EPA AQI, assigning districts via PostGIS, detecting anomalies by z-score, and writing results idempotently to TimescaleDB. Five Airflow DAGs orchestrate historical backfill, weather enrichment, SARIMA forecast recomputation, data quality checks, and FIRMS loading.

The **storage layer** (TimescaleDB + PostGIS on PostgreSQL 16) uses hypertables with seven-day chunk intervals and three continuous aggregates (`aq_hourly`, `aq_daily`, `valley_daily`) that reduce dashboard query latency from seconds to milliseconds. The **API and presentation layer** consists of a FastAPI backend serving ten REST endpoints and one WebSocket endpoint, and a React 18 frontend rendering live station markers on Mapbox GL, Recharts gauges and time-series charts, a D3.js calendar heatmap, and a 72-hour forecast panel.

4.3.3 Key Algorithms

EPA AQI Interpolation. AQI for each pollutant is computed using the standard EPA linear interpolation formula [9]:

$$\text{AQI} = \left(\frac{\text{AQI}_{\text{high}} - \text{AQI}_{\text{low}}}{C_{\text{high}} - C_{\text{low}}} \right) \times (C - C_{\text{low}}) + \text{AQI}_{\text{low}} \tag{4.1}$$

The composite station AQI is the maximum AQI across all pollutants for the same time bucket.

IDW Spatial Interpolation. AQI at each grid point is estimated by inverse distance weighting [11, 12]:

$$\hat{Z}(x_0) = \frac{\sum_{i=1}^{n} w_i \cdot Z(x_i)}{\sum_{i=1}^{n} w_i}, \quad w_i = \frac{1}{d(x_0, x_i)^2} \tag{4.2}$$

The computation is fully NumPy-vectorised, completing in under 50 ms.

SARIMA Forecasting. A SARIMA$(2,1,2)(1,1,1)_{24}$ model is fit to 90 days of hourly AQI data using Python `statsmodels` [13, 14]. The 24-period seasonal order captures the diurnal traffic-driven cycle. Open-Meteo weather variables are included as exogenous covariates, producing a 72-hour forecast with 80% prediction intervals. The system falls back to a persistence model on non-convergence.

5. Expected Outcome

Upon successful completion, HimalayaAir will deliver the following tangible artefacts:

A fully operational real-time pipeline ingesting $PM_{2.5}$, PM_{10}, NO_2, O_3, and CO readings with end-to-end latency under ten minutes, running autonomously on a personal laptop via Docker Compose.

A multi-year historical archive of over 100,000 idempotently ingested readings spanning January 2022 to the present, enriched with weather data and pre-aggregated into three continuous aggregate views for sub-second query performance.

A 72-hour forecasting service maintained by an hourly Airflow DAG, with retrospective MAE and RMSE accuracy metrics stored in a `forecast_accuracy` table.

An interactive public dashboard accessible at `http://localhost:3000` featuring a live Mapbox GL map with AQI-coloured station markers and IDW heatmap overlay, Recharts gauges and time-series charts updated via WebSocket, a D3.js calendar heatmap over three years of valley data, a forecast panel with confidence band shading, and a demo mode for presentation without live data dependency.

A documented portfolio artefact comprising a GitHub repository with a session CHANGELOG, phase completion summaries, and a TimescaleDB benchmark report — directly transferable as a methodology to any city with OpenAQ coverage in the developing world.

References

[1] World Health Organization, *Global Air Quality Guidelines: Particulate Matter ($PM_{2.5}$ and PM_{10}), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide*. Geneva: WHO, 2021. [Online]. Available: https://www.who.int/publications/i/item/9789240034228

[2] IQAir, "2023 World Air Quality Report: Region and City $PM_{2.5}$ Ranking," IQAir, Goldach, Switzerland, 2024. [Online]. Available: https://www.iqair.com/world-air-quality-

[3] P. Kumar, L. Morawska, C. Martani, G. Biskos, M. Neophytou, S. Di Sabatino, M. Bell, L. Norford, and R. Britter, "The rise of low-cost sensing for managing air pollution in cities," *Environment International*, vol. 75, pp. 199–205, Feb. 2015. doi: 10.1016/j.envint.2014.11.019

[4] M. Badura, P. Batog, A. Drzeniecka-Osiadacz, and P. Modzel, "Evaluation of low-cost sensors for ambient $PM_{2.5}$ monitoring," *Journal of Sensors*, vol. 2018, Art. no. 5096540, Oct. 2018. doi: 10.1155/2018/5096540

[5] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. NetDB Workshop*, Athens, Greece, Jun. 2011, pp. 1–7.

[6] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker, and I. Stoica, "Apache Spark: A unified engine for big data processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, Nov. 2016. doi: 10.1145/2934664

[7] A. Khelifati, M. Khayati, A. Dignos, D. E. Difallah, and P. Cudre-Mauroux, "TSM-Bench: Benchmarking time series database systems for monitoring applications," *Proc. VLDB Endow.*, vol. 16, no. 11, pp. 3363–3376, Jul. 2023. doi: 10.14778/3611479.3611532

[8] B. P. Harenslak and J. de Ruiter, *Data Pipelines with Apache Airflow*. Shelter Island, NY, USA: Manning Publications, 2021. ISBN: 9781617296901.

[9] US Environmental Protection Agency, "Technical Assistance Document for the Reporting of Daily Air Quality — the Air Quality Index (AQI)," EPA-454/B-24-001, Research Triangle Park, NC, USA, 2024. [Online]. Available: https://www.airnow.gov/sites/default/files/2024-01/aqi-technical-assistance-document-2024.pdf

[10] J. S. Apte, J. D. Marshall, A. J. Cohen, and M. Brauer, "Addressing global mortality

from ambient $PM_{2.5}$," *Environmental Science & Technology*, vol. 49, no. 13, pp. 8057–8066, Jun. 2015. doi: 10.1021/acs.est.5b01236

[11] D. Shepard, "A two-dimensional interpolation function for irregularly-spaced data," in *Proc. 1968 ACM National Conference*, New York, NY, USA, 1968, pp. 517–524. doi: 10.1145/800186.810616

[12] J. Li and A. D. Heap, "A review of comparative studies of spatial interpolation methods in environmental sciences: performance and impact factors," *Ecological Informatics*, vol. 6, no. 3–4, pp. 228–241, May–Jul. 2011. doi: 10.1016/j.ecoinf.2010.12.003

[13] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ, USA: Wiley, 2015.

[14] U. A. Bhatti, Y. Yan, M. Zhou, S. Li, A. Bhatti, W. Bazai, H. Mirza, and M. A. Bhatti, "Time series analysis and forecasting of air pollution particulate matter ($PM_{2.5}$): An SARIMA and factor analysis approach," *IEEE Access*, vol. 9, pp. 41019–41031, Feb. 2021. doi: 10.1109/ACCESS.2021.3060744

[15] D. Putero, P. Cristofanelli, R. Sprenger, R. Mazot, A. Calzolari, G. P. Gobbi, F. Verza, M. Vuillermoz, and P. Bonasoni, "Influence of open biomass burning on $PM_{2.5}$ and ozone variability in the Kathmandu Valley," in *Atmos. Chem. Phys. Discuss.*, 2015. doi: 10.5194/acpd-15-18733-2015

[16] A. R. Sharma, A. Kharol, K. Badarinath, and D. Singh, "Impact of agriculture crop residue burning on atmospheric aerosol loading — a study over the Indo-Gangetic Plain," *Annales Geophysicae*, vol. 28, pp. 367–379, 2010. doi: 10.5194/angeo-28-367-2010

[17] D. Giri, K. Murthy, and P. Adhikary, "The influence of meteorological conditions on PM_{10} concentrations in Kathmandu Valley," *International Journal of Environmental Research*, vol. 2, no. 1, pp. 49–60, 2008.